Conditionals, Loops, and User-Defined Functions

A Line-by-Line Walkthrough of the COE158 Activity Sheet

Contents

About This Worksheet

This worksheet moves from simple, top-to-bottom scripts into programs that actually **make decisions** and **repeat themselves** — conditional statements, loops, and user-defined functions. These three ideas are the backbone of almost every real program you'll ever write, in MATLAB or any other language, so it's worth understanding *why* each piece of syntax is written the way it is, not just copying it. Each part of the worksheet is explained below in order, followed by assignments at the end.

---

1. `input()` — Getting a Value from the User

```
marks = input('Enter your mark: ');
```

`input` pauses the program, displays the text you give it as a prompt, and waits for the user to type a value and press Enter. Whatever they type is stored in the variable on the left — here, `marks`. By default, `input` expects a **number**; if you wanted the user to type text instead, you'd add `'s'` as a second argument (`input('Enter your name: ', 's')`).

Why it's used here: it's what makes the program **interactive** instead of hardcoded. Without `input`, you'd have to manually edit the script and change `marks = 65;` every time you wanted to test a different value — `input` lets the same script run differently each time, without touching the code.

---

2. `if ... else ... end` (Part A)

```
if marks >= 50
    disp('PASS')
else
    disp('FAIL')
end
```

This is the simplest form of decision-making in MATLAB. `if marks >= 50` is a **condition** — an expression that evaluates to either true or false. If it's true, MATLAB runs the code between `if` and `else`; if it's false, it skips straight to the code between `else` and end. Either way, only **one** of the two branches ever runs, never both.

Note the **indentation** of `disp('PASS')` and `disp('FAIL')` — MATLAB doesn't strictly require it to work, but indenting the code inside a block makes it dramatically easier to read, especially once your conditions get more complicated. Also note that every `if` must be closed with a matching end.

Why it's used here: this models the most basic kind of engineering/academic decision — "does this value clear a threshold, yes or no?" It's deliberately the simplest possible example before the worksheet adds more branches.

---

3. `if ... elseif ... else ... end` (Part B)

```
if marks >= 80
    disp('Excellent')
elseif marks >= 70
    disp('Very Good')
elseif marks >= 60
    disp('Good')
elseif marks >= 50
    disp('Pass')
else
    disp('Fail')
end
```

`elseif` (written as one word) lets you chain together **more than two** possible outcomes. MATLAB checks each condition **from top to bottom** and stops at the very first one that's true — it does not keep checking after that. This is why the conditions are written in **descending order** (80, then 70, then 60, then 50): if they were written

in ascending order instead, a mark of 95 would incorrectly match `marks >= 50` first and print "Pass," since MATLAB would never get as far as checking `>= 80`.

Why it's used here: real grading, and most real engineering decisions, rarely have just two outcomes. This teaches you to think about the **order** conditions are checked in, which is a common source of bugs for beginners.

---

4. `for` Loop — Basic Counting (Part C)

```
for i = 1:10
    disp(i)
end
```

A `for` loop repeats the code between `for` and `end` once for every value in the range you give it. Here, `i = 1:10` means `i` takes the values 1, 2, 3, ... up to 10, one at a time, running `disp(i)` on each pass. The loop variable `i` is just a regular variable — you could name it anything — but `i`, `j`, `k` are conventional names for loop counters in most programming languages, MATLAB included.

Why it's used here: it's the clearest possible demonstration that a `for` loop runs a **known, fixed number of times** — exactly 10, because the range `1:10` has exactly 10 values in it.

---

5. `for` Loop with Calculations (Part D)

```
number = input('Enter a number: ');
for i = 1:12
    result = number * i;
    fprintf('%d x %d = %d\n', number, i, result);
end
```

This combines what you already know: `input` gets a starting number, and the `for` loop runs 12 times (`i` from 1 to 12), calculating `result = number * i` fresh on every pass — this is exactly how you'd generate a multiplication table by hand, just automated.

`fprintf` and format specifiers: unlike `disp`, `fprintf` lets you build a formatted line of output with values inserted into specific positions:

- `%d` is a placeholder for a **whole number (integer)** — there are three of them here because the line needs to insert `number`, `i`, and `result`, in that order (the order of the `%d`s must match the order of the variables listed after the text).
- `\n` means "move to a new line" — without it, all 12 lines of the multiplication table would run together on a single line.

Why it's used here: `fprintf` is introduced specifically because `disp` can't easily mix fixed text and multiple numbers on one line the way this table needs — you'd have to manually build a string with square brackets and `num2str` for every value, which gets messy fast with three numbers on one line.

---

6. `while` Loop (Part E)

```
count = 1;
while count <= 10
    disp(count)
    count = count + 1;
end
```

A `while` loop keeps running its block of code **for as long as** its condition stays true — it doesn't know in advance how many times that will be, unlike a `for` loop. Here, `count` starts at 1, and the loop keeps going as long as `count <= 10`. Crucially, `count = count + 1;` **inside** the loop is what eventually makes the condition false — without that line, `count` would stay at 1 forever and the condition `1 <= 10` would never stop being true, creating an **infinite loop** (you'd have to press Ctrl+C to force it to stop).

Why it's used here: it's deliberately built to produce the exact same output as the `for` loop in Part C, so you can directly compare the two approaches. This is the worksheet's way of showing that a `for` loop and a `while` loop can sometimes do the same job, but a `while` loop requires *you* to manage the counter yourself, both creating it beforehand and updating it inside the loop.

---

7. Nested Loops (Part F)

```
for i = 1:5
    for j = 1:5
        fprintf('%d ', i*j);
    end
    fprintf('\n');
end
```

A **nested loop** is a loop written inside another loop. Here, for every single value of `i` (the "outer" loop, running 5 times), the entire "inner" loop over `j` runs completely (all 5 values), before `i` moves on to its next value. That means the line `fprintf('%d ', i*j)` actually runs 5 × 5 = 25 times in total.

Look carefully at where each `fprintf` sits: `fprintf('%d ', i*j)` is **inside** the inner loop (so it prints one number, with a trailing space, for every `j`), while `fprintf('\n')` is **inside the outer loop but outside the inner one** — meaning it runs once per value of `i`, moving to a new line only after an entire row has been printed.

Why it's used here: this is the classic way to build a table or grid, one row and one column at a time. Getting the indentation and the placement of the two `end` statements correct is exactly the skill nested loops are testing here — it's easy to accidentally put `fprintf('\n')` in the wrong place and get a single long line instead of a proper table.

---

8. Creating a User-Defined Function (Part G)

```
function area = calculateArea(radius)
    area = pi * radius^2;
end
```

This is a **function file**, and it works differently from every script you've written so far. Reading the first line piece by piece:

- **`function`** — the keyword that tells MATLAB this file defines a function rather than a plain script.
- **`area`** — the name of the **output** the function will return.
- **`calculateArea`** — the function's name. This name **must exactly match the filename** (`calculateArea.m`) — this is a strict MATLAB rule, not just a suggestion.
- **`(radius)`** — the **input** the function expects to be given when it's called.

Inside the function, `area = pi * radius^2;` calculates the result using the input, and because `area` was named as the output back in the first line, that's automatically what gets returned when the function finishes. `pi` is a MATLAB built-in constant equal to π — you never need to define it yourself.

Why it's used here: this is the simplest possible function — one input, one output, one line of calculation — specifically so you can focus on the structural rules (matching filename, naming the output correctly) without the maths getting in the way.

---

9. Calling a User-Defined Function (Part H)

```
r = input('Enter the radius: ');
A = calculateArea(r);
fprintf('Area = %.2f\n', A);
```

Once `calculateArea.m` is saved in the **same folder** as this script, you can call it exactly like any built-in MATLAB function: `calculateArea(r)` runs the function using whatever value is stored in `r`, and the result is stored in `A`.

`%.2f` in `fprintf`: this is a new format specifier — `f` means "format as a decimal (floating-point) number," and `.2` means "show exactly 2 digits after the decimal point." This matters here because `pi * radius^2` will usually produce a long, messy decimal (e.g. `78.53981633974483`) — `%.2f` rounds that down to a clean, readable `78.54` for display purposes only (the full-precision value is still stored in `A` itself).

Why it's used here: this demonstrates the entire point of writing a function in the first place — once it's saved, you (or a teammate) can reuse `calculateArea` in as many different scripts as you like without ever rewriting the formula.

---

10. Functions with Multiple Inputs (Part I)

```
function P = calculatePower(V, I)
    P = V * I;
end
```

This works exactly like `calculateArea`, but the parentheses now list **two** inputs, `V` and `I`, separated by a comma. When you call it — `calculatePower(voltage, current)` — the values are matched up **by position**: whatever you pass first becomes `V` inside the function, and whatever you pass second becomes `I`, regardless of what you happened to name those variables in your calling script.

Why it's used here: most real engineering formulas depend on more than one quantity (here, power = voltage × current). This shows that adding another input is as simple as adding another name inside the parentheses — the structure of the function doesn't otherwise change.

---

11. Combining Everything (Part J)

The practice exercise deliberately requires you to combine every idea from this worksheet into a single program:

- A **`for` loop** to ask for five scores one at a time (reusing `input` inside the loop, once per iteration)
- **Conditional statements** (`if...elseif...else`) applied to *each* score inside that same loop, to work out its grade
- A separate **user-defined function**, `calculateAverage.m`, to compute the average — kept as its own function rather than written inline, for the same reusability reason as Parts G–I
- **`fprintf`** or `disp` to present the scores, the average, and each grade clearly

The key design decision worth noticing: the loop that **collects and grades** the scores, and the **function** that averages them, are two separate jobs. You could technically calculate the average inside the loop too, but keeping it as its own function is better practice — if you later needed the average calculated somewhere else in a bigger program, you'd already have a reusable piece ready to call, exactly like `calculateArea` and `calculatePower`.

Summary — Quick Reference

Concept	Syntax	Purpose
Get user input	`x = input('prompt: ')`	Pause and read a value typed by the user
Two-way decision	`if cond ... else ... end`	Run one of two blocks depending on a condition

Concept	Syntax	Purpose
Multi-way decision	`if ... elseif ... else ... end`	Check several conditions in order, top to bottom
Counted loop	`for i = a:b ... end`	Repeat a known number of times
Conditional loop	`while cond ... end`	Repeat until a condition becomes false (you manage the counter)
Nested loop	`for ... for ... end ... end`	A loop inside a loop; runs rows × columns times
Formatted output	`fprintf('%d %s %f\n', ...)`	Print text and numbers together, with control over layout
Integer placeholder	`%d`	Insert a whole number into an `fprintf` string
Decimal placeholder	`%.2f`	Insert a decimal number, rounded to 2 places
New line	`\n`	Move to the next line in `fprintf` output
Function definition	`function out = name(in) ... end`	Package reusable logic; filename must match function name
Calling a function	`result = name(value)`	Run a saved function using a given input
Multiple inputs	`function out = name(in1, in2) ... end`	Inputs matched by position when the function is called

Big ideas to take away:

1. `if`/`elseif` conditions are checked **in order** — put the most specific or highest condition first.
2. Use a `for` loop when you know exactly how many repeats you need; use a `while` loop when you're repeating "until something becomes true."
3. A `while` loop's stopping condition must be updated **inside** the loop, or it will never end.
4. A function file's **filename must exactly match** the function name inside it.
5. Functions take inputs **by position**, not by name — the order you pass values in matters.

Assignments and Practice Examples

Assignment 1 — Trace the Grades

Without running MATLAB, work out by hand what each of these scripts from Parts A

and B would print for `marks = 45`, `marks = 62`, and `marks = 100`. Then run them for real and check your predictions.

Assignment 2 — Swap the Loop Type

Rewrite the multiplication-table program from Part D (`number * i` for `i = 1:12`) so that it uses a **`while` loop** instead of a `for` loop, producing exactly the same output. Compare your version with the original and identify every extra line you had to add.

Assignment 3 — Extend the Nested Loop

Modify the nested loop from Part F so that instead of a 5×5 multiplication table, it produces a **10×10** table. Then modify it again so each number is padded to line up neatly in columns (hint: look up the `%4d` format specifier inside `fprintf`, and think about where the space in `'%d '` needs to change).

Assignment 4 — Write Your Own Function

Write and save a new function called `calculateBMI.m` that takes two inputs, `weightKg` and `heightM`, and returns a person's Body Mass Index using the formula `BMI = weight / height^2`. Then write a separate script that asks the user for their weight and height using `input`, calls your function, and displays the result using `fprintf` with two decimal places.

Assignment 5 — Full Combination Task

Complete Part J exactly as described in the worksheet: collect five scores with a `for` loop, grade each one with conditional statements, calculate the average using a separate `calculateAverage.m` function, and display all scores, the average, and each grade. Once it works, add one more feature of your own: display the **highest** and **lowest** of the five scores as well.

Assignment 6 — Reflection

In your own words, answer:

1. Why does the order of conditions matter in an `if...elseif...else` chain, but not in a simple `if...else`?
2. Give an example (other than the ones in this worksheet) of a task better suited to a `while` loop than a `for` loop, and explain why.
3. What would happen if you saved the function from Part G under the filename `area.m` instead of `calculateArea.m`? Try it and describe what happens when you attempt to call it.